

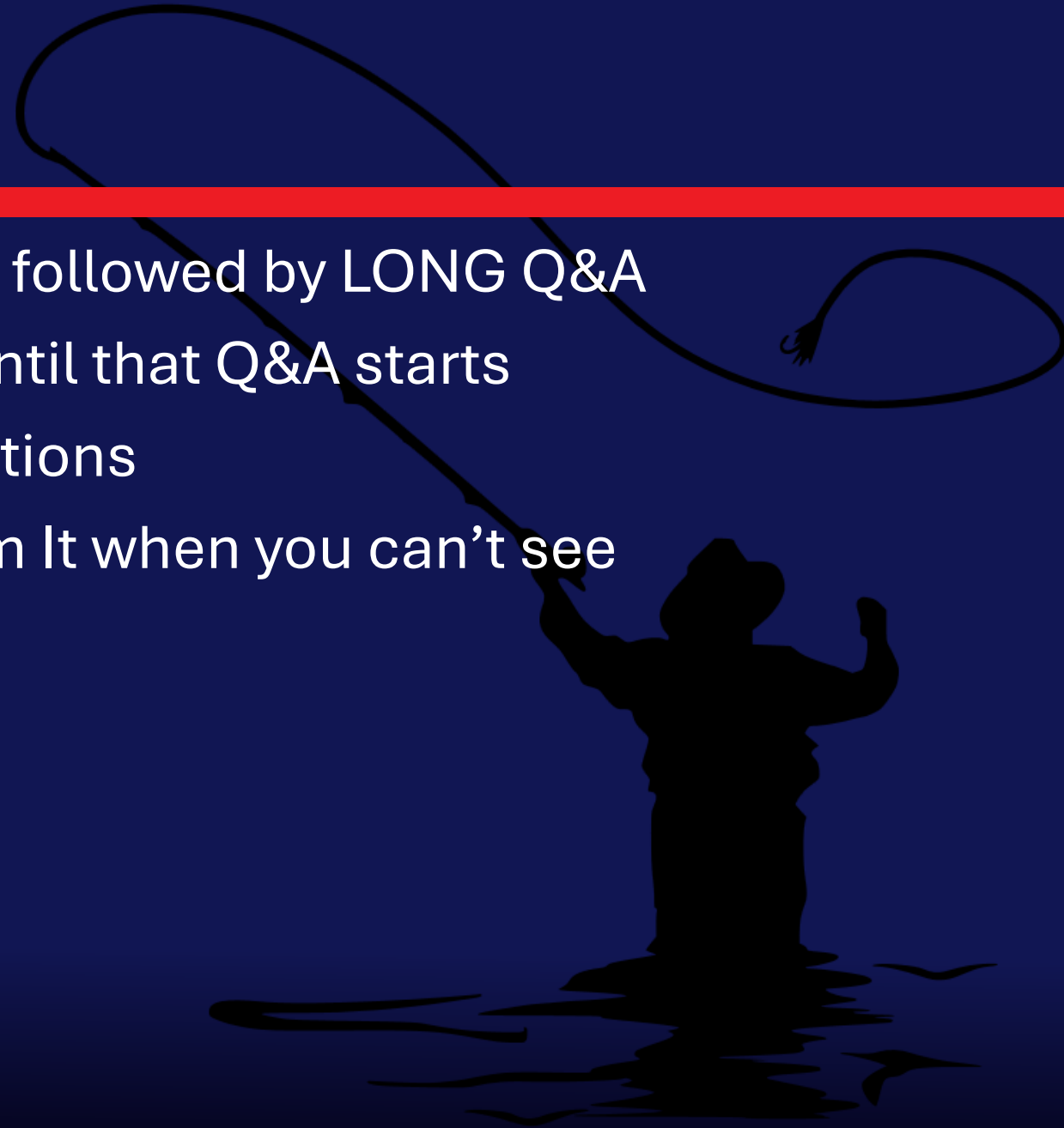
Don't Brick the CEO's Mac

Building and Automating macOS Labs for Risk-Free Policy Testing



Attention

- ◆ MMS sessions are 60-75 minutes followed by LONG Q&A
- ◆ Please hold detailed questions until that Q&A starts
- ◆ Feel free to ask clarification questions
- ◆ Remind the speakers to use Zoom It when you can't see



Speakers



Frank Lesniak

Corporate M&A Focused

On all the socials @

← <https://linktr.ee/franklesniak>



Michael Niehaus

Programme Director



mtniehaus



[in/mniehaus](https://www.linkedin.com/in/mniehaus) →



Can't see our slides? Can't hear? Need to repeat the question? Call us out!

The Executive's Laptop

The Scenario

A closed laptop sits on an executive desk. The FileVault unlock screen glows. Where the recovery key would appear, there is only a black redaction bar. The helpdesk phone rings. Then the director's phone rings.

You would never ship a BitLocker policy to the CEO's Windows laptop without testing recovery. Yet many organizations ship FileVault, PPC, and Defender to Macs with a fraction of that confidence.

The Notification Stack

-  Missed call from CFO
-  "Board call in 30 min"
-  *"My laptop will not let me in."*

- ⊗ Production is a terrible place to discover your Mac policy assumptions.

The cost is not just the laptop — it is every calendar event the laptop owns: the board call, the keynote, the quarter-close approval.

Session Takeaways

1

Analyze Virtualization Constraints

Analyze the virtualization constraints of Apple Silicon and select the appropriate hypervisor — Parallels vs. UTM — that aligns with your budget and automation needs.

2

Construct an Automated Lab

Construct a fully automated, reproducible macOS test lab using PowerShell 7 to fetch restore images and control VM states.

3

Execute End-to-End Validation

Execute end-to-end validation of high-risk policies — FileVault, Defender — by enrolling, breaking, and rolling back VMs via script, keeping production safe.

4

Implement the GitHub Starter Kit

Implement the provided GitHub starter kit immediately to bridge the gap between your Windows automation skills and macOS requirements.

Agenda

01

Why This Matters

Risk map and Windows translation

02

Constraints

What drives lab design on Apple Silicon

03

Choose Your Hypervisor

Parallels vs. UTM vs. Tart

04

The Automation Layer

The PowerShell module and provider model

05

Live Demos

Pin. Build. Swap. Break. Roll back. Prove it.

06

Wrap-Up

Troubleshooting, the kit, Monday plan, recap

07

Extended Q&A

30 minutes

08

Thank-You and Close

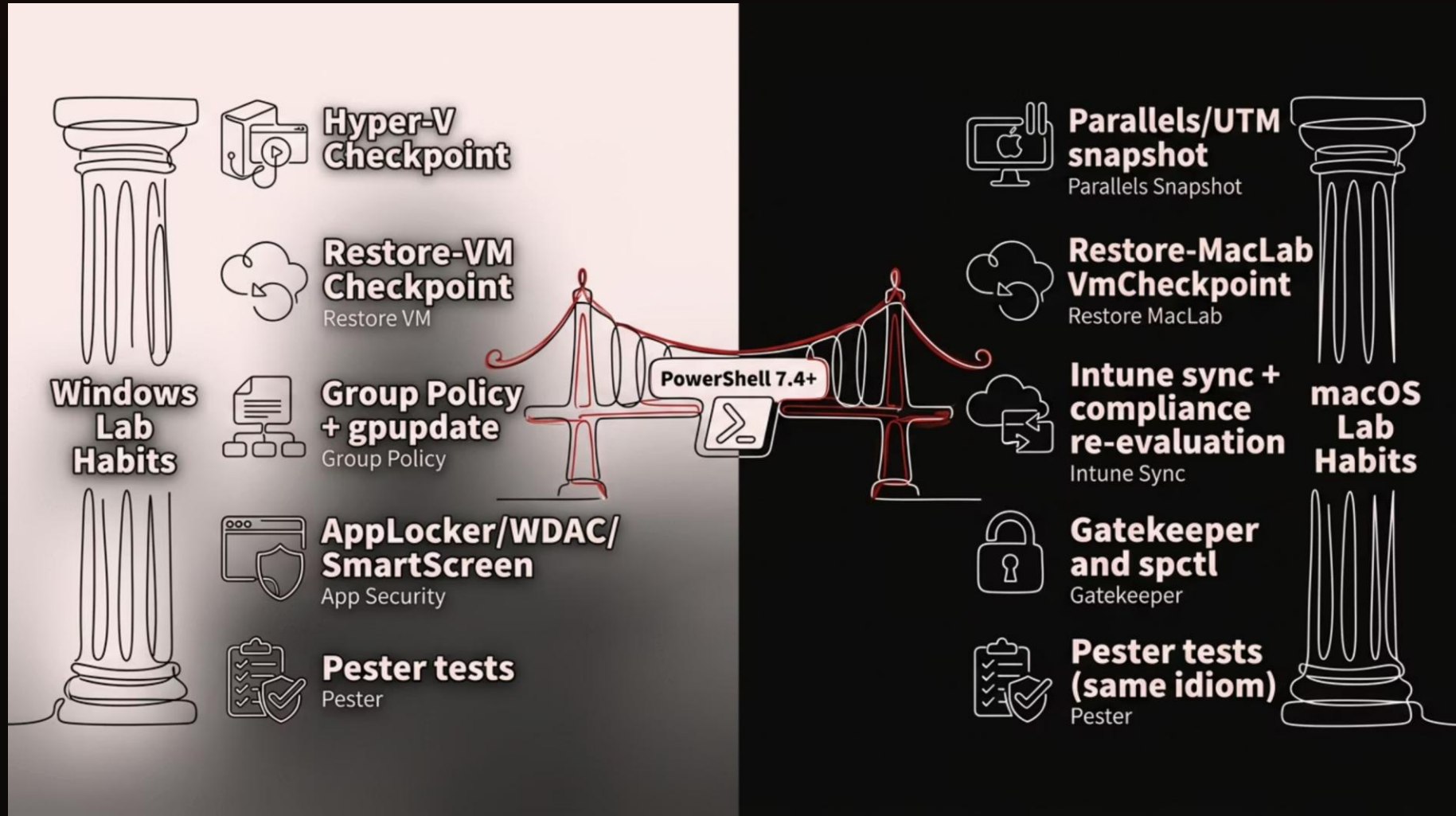
The Risk Map

Every one of these risk categories has a calendar event attached. The goal is to make sure that event is a test run, not a Monday morning incident.

Risk	What Breaks	First to Notice	Why Leadership Cares
 FileVault	Unlock, recovery, escrow, prompts	CEO locked out → helpdesk → director	Board call starts in 30 minutes
 PPC / TCC	Screen recording, EDR, remote support	User, often in front of customers	Support tools go blind; trust erodes
 Defender	System ext, Full Disk Access, onboarding	SOC, from missing telemetry	Security posture gap; compliance risk
 App Execution Control	Signed, notarized apps blocked	Developer → manager	Engineering stops shipping
 Compliance / CA	Outlook, SharePoint, Teams, approvals	Whoever is sending the invoice	Revenue-cycle interruption

Where Your Windows Instincts Already Apply

Mac testing is similar to Windows – just with different tools



✓ Your Windows lab grammar is still valid: checkpoint → apply → observe → roll back → document. Only the instruments change.

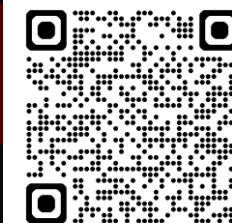
Translation Cheat Sheet

You do not need new instincts. You need a translator. The full cheat sheet lives at `docs/Windows-Admin-Cheat-Sheet.md` in the repo.

Windows	macOS Lab Equivalent	Point to Make
BitLocker	FileVault	Recovery key escrow, test encryption & escrow
Hyper-V checkpoints	Parallels / UTM snapshots	Same restore discipline; different API
Group Policy + gpupdate	Intune sync + compliance re-eval	Cloud timing varies; test with checkpoints
AppLocker / WDAC / SmartScreen	Gatekeeper and spctl	Policy shape differs; break-&-rollback is identical
Defender health (Windows)	mdatp health	Capture as evidence; not just "looks installed"
Pester tests	Pester tests	Same idiom, same patterns — zero relearning

📄 The repo carries the complete table. Don't copy it — clone the kit.

<https://github.com/franklesniak/macOSLab>



Constraints That Drive Lab Design:

What is true on Apple Silicon – and what that means for your lab.

⚠ NOTE: Things we are **not** promising today: ADE/ABM zero-touch, Platform SSO unlock, Touch ID, Secure Enclave-dependent behavior, executive pilot sign-off. Those still require real hardware.



Apple Silicon Is Not Hyper-V

Windows Lab

Hardware → Hyper-V / VMware → Guest

- Any guest OS
- Bridged networking everywhere
- Snapshot anything, anytime
- Broad cross-version guest support

macOS Reality on Apple Silicon

Hardware → Host macOS → Apple Virtualization Framework → macOS Guest

- macOS guests on Apple Silicon use Apple's Virtualization framework
- Parallels, UTM, and Tart all sit on top of it
- Same-major host/guest is the safest default
- Some host integrations do not apply in VM context

Key takeaway: Treat your lab Mac like a build server. Keep its major version tied to its macOS VMs' major version.

Fleet Coverage Drives Host Architecture

Best practice: pin your host Mac to the same major version of macOS as your VMs' major version

macOS Sonoma 14.x

Example Laptop (used/refurb):

MacBook Pro, **M2** Max
64+ GB RAM, 2+ TB Storage

If needed, could be repurposed to a macOS Sequoia 15.x or macOS Tahoe 26.x host

Example "Desktop" (used/refurb):

Mac Mini (2023), **M2** Pro
32 GB RAM, 2+ TB Storage

Note: 32 GB of RAM limits you to ~1 VM

If needed, could be repurposed to a macOS Sequoia 15.x or macOS Tahoe 26.x host

macOS Sequoia 15.x

Example Laptop (used/refurb):

MacBook Pro, **M4** Pro (or Max)
64+ GB RAM, 2+ TB Storage

If needed, could be repurposed to a macOS Tahoe 26.x host

Example "Desktop":

Mac Mini, **M4** Pro
64+ GB RAM, 2+ TB Storage

If needed, could be repurposed to a macOS Tahoe 26.x host

macOS Tahoe 26.x

Example Laptop:

MacBook Pro, **M5** Pro (or Max)
64+ GB RAM, 2+ TB Storage

Note: not backward-compatible with older versions of macOS

Example "Desktop":

Mac Mini, **M4** Pro
64+ GB RAM, 2+ TB Storage

If needed, could be repurposed to a macOS Sequoia 15.x host

⚠ Three supported macOS majors can mean three pinned hosts. One big Mac may not cover every version in your fleet.

Licensing: The Two-Guest Boundary

What Apple's License Permits

Apple's macOS software license permits limited additional virtual instances of macOS on Apple-branded hardware for permitted purposes — software development, testing during development, macOS Server, and personal non-commercial use are typical examples.

For this deck and starter kit, design for **no more than two concurrent macOS guests per Apple-branded host** unless your organization verifies a different license posture with legal and procurement.

The Engineering Implication

You need Apple hardware to run macOS in a virtual machine!

If you need wider coverage, the answer is more hosts — not bigger hosts.

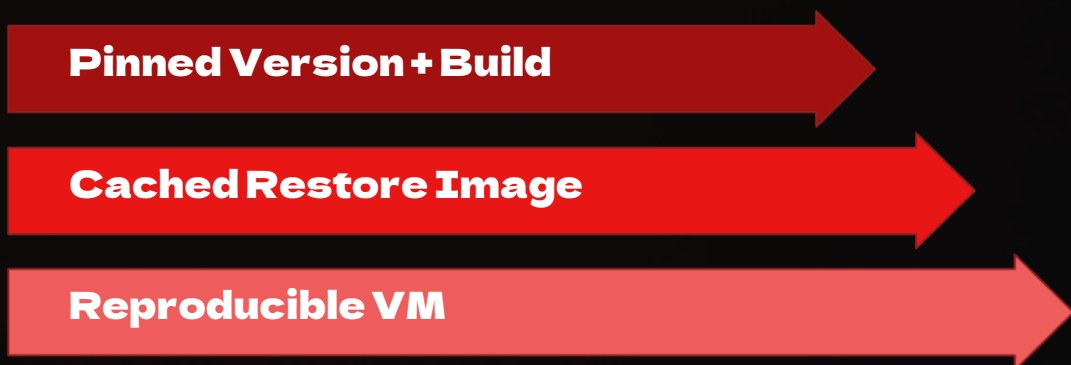
The licensing story is friendlier than Windows licensing in some ways. The engineering constraints are where the real design work starts.

 This is not legal advice. Verify your organization's interpretation before implementing/scaling your VMs.

Pin the Build, Cache the Artifact

If you cannot say which macOS build you tested, you cannot say what your test means.

The Reproducibility Flow



The Media Cache Structure

```
~/Demo/Installers/  
UniversalMac_26.4.1_25E253_Restore.ipsw  
26.4.1-25E253.metadata.json  
26.4.1-25E253.mist-download.json
```

- Artifact type: IPSW (restore image), not bootable disk image
- SHA-256 checksum recorded in metadata sidecar
- Cache reused across runs — never re-download on venue Wi-Fi
- Media stored outside `./_evidence` tree

✓ Every change-board ticket can now answer: "Which build did you test on?"

The Fidelity Traffic Light

● Green – VM Is Sufficient

- Profile receipt and delivery confirmation
- Gatekeeper and spctl assessment
- Defender health checks
- Rollback routines and evidence export

Today's demo lives here.

● Yellow – VM Iteration, Then Physical Sign-Off

- FileVault rollout behavior end to end
- Recovery-key process and user prompts
- Performance-sensitive Defender behavior

● Red – Physical Hardware Required

- ADE/ABM zero-touch enrollment
- Platform SSO sign-in and unlock
- Touch ID and Secure Enclave behavior
- Final executive pilot sign-off

Stage Rig vs. Operating Lab

Dimension	What We Show Today	What You Run After the Talk
Purpose	Prove the workflow and cadence	Coverage, cadence, and evidence
Scope	One known-good 26.x stage host	Host coverage aligned to fleet cohorts
Host Strategy	Mac Mini M4, M4 Pro, 64 GB RAM, 4 TB storage macOS 26.x, prepared IPSW	Potentially multiple Macs, each supporting a different major version of macOS
Network / Cloud	Screenshots and recording fallback	Provider version matrix; repeatable media cache
Evidence	Live run output, fixture-backed fallback	Evidence bundles per run, committed to change record
Hardware Sign-Off	Not required for today's demo	Required for “yellow” and “red” fidelity items

 Don't buy one large Mac and assume it covers every fleet risk. Build a coverage plan tied to your Mac population and upgrade cadence.

PART 2 OF 4

Choose Your Hypervisor:

Choose by operating model, not by preference.

The Hypervisor Decision

We are not picking the best product. We are picking the operating model that makes safe behavior easiest for your team.

Parallels Desktop Pro/Business

Polished commercial workflow. Best fit when you need reliable automation, a procurement-friendly line item, and the most dependable live-demo story. This is the primary demo path today.

UTM

Free experimentation path. Best fit when there is no tool budget and a documented manual step for VM creation and checkpointing is acceptable. Open-source, honest about its gaps.

Tart

CLI-first automation, future CI. Best fit when automation is the long-term goal and the team is ready for a roadmap conversation. Stubbed in v1 of the starter kit.

- ❏ VMware Fusion is not in the core comparison — current Broadcom documentation does not support Arm macOS as a Fusion guest on Apple Silicon. It remains relevant for Arm Windows or Linux VMs.

The Decision Matrix

Parallels for polish. UTM when budget is the constraint. Tart when automation is the goal.

Dimension	Parallels (prlctl)	UTM (utmctl + template)	Tart (tart)
Cost	Paid subscription	Free / open source	Fair Source; free-tier CPU limit
Admin UX	GUI + full CLI	GUI + partial CLI	CLI-first, no GUI
Automation Surface	prlctl, full lifecycle	utmctl: list/start/stop proven	tart, automation-native
Snapshots & Clones	✓ Full scripted support	– Manual step required in v1	✓ Scripted
Provider Guardrails	Hardening verified post-create	Manual template + doc step	Isolation by design
Stage Reliability	✓ Safest live demo bet	Partial — use Parallels for stage	* Future path
Procurement Story	Easy line item	No cost; open license	Roadmap conversation
CI Path	Not primary	Not primary	✓ Built for CI

Note: Tart is the single-host CLI. Orchard is Cirrus Labs' orchestration layer for fleets of Tart-capable hosts — not a fourth provider.

PART 3 OF 4

The Automation Layer: PowerShell provides a familiar interface.

Windows PowerShell

Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell <https://aka.ms/pscore6>

PS C:\Users\Administrator> Get-Process | Sort-Object CPU -Descending | Select-Object -First 5

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
4624	196	145976	162732	12.48	1234	1	powershell
3167	157	102344	110548	7.23	5678	1	YourApp
2715	123	82344	91204	5.11	9101	1	chrome
1889	98	51284	60312	3.09	1122	1	explorer
1536	78	30412	35488	1.23	3344	1	svchost

PS C:\Users\Administrator> Get-Service | Where-Object {\$_.Status -eq 'Running'}

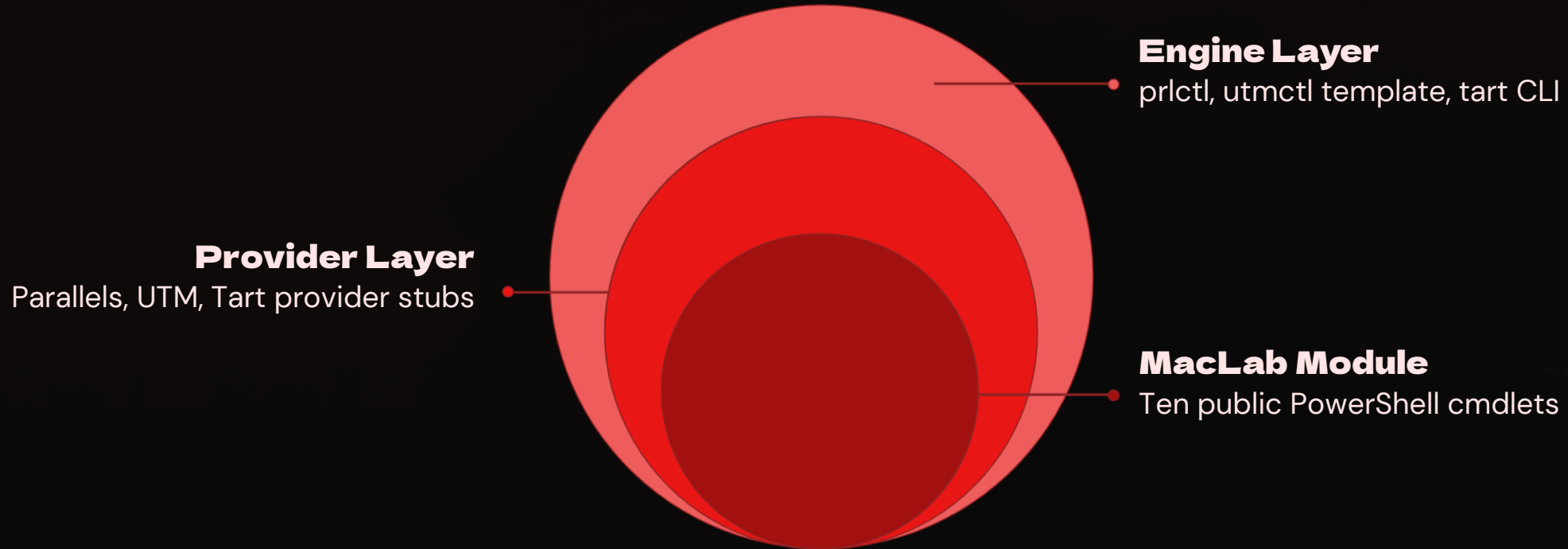
Status	Name	DisplayName
Running	WinDefend	Windows Defender Antivirus Service
Running	wuauserv	Windows Update
Running	bits	Background Intelligent Transfer Service
Running	LanmanServer	Server
Running	Schedule	Task Scheduler
Running	EventLog	Windows Event Log
Running	Dhcp	DHCP Client
Running	DnScache	DNS Client
Running	Spooler	Print Spooler
Running	W32Time	Windows Time

PS C:\Users\Administrator> Get-ComputerInfo | Select-Object WindowsProductName, OsVersion, CsName

WindowsProductName	OsVersion	CsName
Windows 10 Pro	10.0.19045	WORKSTATION-01

PS C:\Users\Administrator>

One Operator Interface, Three Engines



The Abstraction Promise

One operator interface — the public cmdlets — sits above three provider implementations. Windows admins can adopt the kit because the cmdlet names use verbs they already know.

What It Does Not Hide

The abstraction surfaces capability gaps with explicit `ManualStepRequired` errors when a provider cannot automate a step. Differences are documented, not papered over.

The Lifecycle Flow

Step 12 is the difference between a repeatable lab and stale records. The loop arrow is the entire point — you run this 50 times, you push to production once.

Build the Lab



Prepare Test State



Prove and Recover



⊗ Cloud state does not roll back. Intune, Entra, Defender portal state, audit logs, and assignments are not restored by a VM snapshot restore.

Snapshot Taxonomy: Five Names, Five Purposes

Snapshot time travel is real. The cloud does not time-travel with it. Use the same vocabulary in the runbook, cmdlets, and docs — consistency eliminates ambiguity during incidents.

Clean-OS

Fresh OS install, no enrollment.
Baseline for VM reuse. *Use when:* starting a new test sequence from scratch.

Pre-Enroll

Clean OS, ready for enrollment.
Enrollment retry point. *Use when:* you need to re-run the enrollment step without rebuilding the VM.

Post-Enroll-Baseline

Enrolled, policies in good state. Speed baseline. **Demo 4 starts here.** *Watch out for:* cloud policy drift after capture.

Broken-Policy-State

Engineered failure state. **Stage failures should look planned because they are.** *Use when:* live Intune sync did not land in time for the demo.

Recovered-Known-Good

Captured after rollback AND after report-only cloud cleanup review or manual reconciliation notes. Not just after the local restore.

Demo Rules and the Cloud-State Warning



Rule 1: Downloads Are Cached

No live mist download. The artifact is pinned, cached, and recorded. We skip ahead when the educational part is over. Watching a progress bar is not educational.



Rule 2: Cloud Sync Is Not Instant

Checkpoints exist for a reason. The live Intune stage thread may or may not have landed. We have fallbacks for every outcome.



Rule 3: Failures Are Prepared

They should look planned because they were. Broken-Policy-State is engineered, not improvised. Stage failures are rehearsed scenarios.



Rule 4: Rollback Restores the VM, Not Intune

Snapshot rollback restores the VM. It does **not** rewind Intune, Entra, Defender portal state, audit logs, or assignments.



Snapshot rollback restores the VM. It does not rewind Intune, Entra, Defender portal state, audit logs, or assignments.

PART 4 OF 4

The Demos:

Pin. Build. Swap. Break.

Roll back. Prove it.



Demo 1: Pin and Acquire Media

Reproducibility starts the moment you pin the build. This is what you commit to your lab change record.

What We Are Doing

- Pin macOS 26.4.1 build 25E253
- Verify the prepared IPSW artifact against SHA-256
- Write the media metadata sidecar

What Success Looks Like

- Prepared artifact reused (no re-download)
- SHA-256 verified against cached value
- `26.4.1-25E253.metadata.json` produced

Script Output

```
Demo1-Media.ps1
MediaId      26.4.1-25E253
ArtifactType Firmware
Prepared     True
MetadataJson ~/Demo/Installers/
              26.4.1-25E253
              .metadata.json
```

The metadata sidecar records: version, build, source, artifact type, cache root, acquisition time, artifact path, SHA-256, and prepared-artifact flag.

Demo 2: Parallels VM and Snapshot

Provider commands return 0. That is not the same as being safe. Hardening is verified after creation, not assumed from the exit code.

What We Are Doing

- Verify the same prepared IPSW artifact
- Create the Parallels VM with the default Baseline sizing profile
- Run provider hardening checks: shared profile, SmartMount, shared clipboard, device auto-sharing, host location
- Capture Clean-OS checkpoint with clean-shutdown requirement

What Success Looks Like

- VM record includes provider version and isolation state
- Clean-OS checkpoint record includes the provider snapshot ID
- Media record points back to the metadata sidecar

⚠ At the end of this segment, we'll start Demo 4: restore Post-Enroll-Baseline, confirm Firefox is installed, assign or sync the Gatekeeper policy, run Company Portal Check Status, then park the VM while Demo 3 runs.

Provider Hardening Checklist*

- Shared profile — disabled
- App sharing — disabled
- SmartMount — disabled
- Shared clipboard/cloud — disabled
- Device auto-sharing — disabled
- Host location access — disabled

Demo 4 Setup: An Audit Finding

Finding: User-Installed Apps from Non-App-Store Sources

A synthetic audit memo recommends hardening Gatekeeper to App Store sources only. A reasonable admin tightens the policy. The CEO opens a line of business app (we're using Firefox as a simple example). The CEO calls.

Good intentions can still create executive outages. The lab is where this scenario is supposed to surface — not on the executive's laptop on a Monday morning.

- ⊗ Reasonable security recommendation. Reasonable admin response. Unreasonable Monday morning. 😊

Why Gatekeeper for the Live Break?

- Shows the break-and-rollback loop without recovery-key handling on a projector
- Same loop applies to FileVault and Defender
- Firefox is a universally recognizable test subject. You could also use VS Code or any other signed application not distributed through the App Store.
- Failure mode is unambiguous and fast to demonstrate

- ⚠ As mentioned on the previous slide, we're doing initial setup for demo 4 ahead of demo 3. This is to allow group membership to bake and the policy change to take effect.

Demo 3: Provider Swap to UTM

The provider abstraction prevents tool differences from changing the entire workflow. UTM has partial automation in v1:

Prepared Manually

- `macos-lab-template.utm.json`
- `macOSLab-UTM-Disposable`
- Apple Virtualization framework
- Shared Network mode

Script Verifies

- Same pinned IPSW metadata (26.4.1, build 25E253)
- `Get-MacLabVm -Provider UTM -Name macOSLab-UTM-Disposable`
- Isolation: no host-sharing features that blur boundaries

Script Reports

- VM record with `TemplatePath`
- `ManualStepRequired` flags where applicable
- Automated: list, status, start, stop
- Manual in v1: create, checkpoint, IP discovery, destructive cleanup

❗ Tart note: if you want this in CI, Tart is probably the right tool. It is stubbed in v1 of the starter kit.

Back to Demo 4... System Policy Control: Two Settings, Big Impact

Two switches in Settings Catalog can lock the CEO out of their line of business app.

Intune Settings Catalog – System Policy Control

Setting	Value
Enable Assessment	✓ Enabled
Allow Identified Developers	✗ Disabled

Result: Only App-Store-source apps will pass spctl assessment. Firefox is identified-developer signed, not App Store distributed — it fails immediately.

- ✗ Notarization is not enough. The policy explicitly excludes identified developers.

The Policy Chain

- Category: System Policy Control (commonly known as Gatekeeper)
- Delivered via Intune Settings Catalog profile
- Assessment is enabled AND identified-developer exception is removed
- Firefox: signed by identified developer, not App Store distributed
- Outcome: blocked by spctl at launch

The live Intune stage thread has been baking since Demo 2. If it landed, use it. If not, Broken-Policy-State and fixture-backed evidence deliver the same payoff.

Demo 4 Live Run

Policy failure belongs in the lab, not on the executive laptop.

State Machine

1	Baseline Firefox installed. All green.
2	Broken Firefox rejected by spctl. Expected FAIL.
3	Rolling Back VM networking disconnected. Restore initiated.
4	Recovered Firefox launches again. PASS row lights up.

Expected Outcomes

PASS	MDM enrollment profile present
PASS	Gatekeeper assessment enabled
PASS	System Policy Control profile detected
FAIL	Firefox blocked — App Store only (expected)
PASS	Blocking dialog captured
PASS	Evidence redaction applied
PASS	Rollback restored Post-Enroll-Baseline
PASS	Firefox accepted after rollback
WARN	Intune cloud assignment requires report-only cleanup review

Anatomy of an Evidence Bundle

Evidence beats portal confidence. This is what your Change Advisory Board actually wants — not "the portal says assigned."

evidence.json (Sanitized Example)

```
{
  "$schemaVersion": "1.0.0",
  "runId": "2026-05-06T14:22:11Z",
  "provider": "Parallels",
  "snapshot": "Post-Enroll-Baseline",
  "fidelity": "Green",
  "redactionApplied": true,
  "intuneDeviceId": "***REDACTED***",
  "providerVersionMatrix": {
    "hostMacOS": "26.4.1 (25E253)",
    "intunePolicySetId": "***REDACTED***"
  }
}
```

Run Output Files

- evidence.json
- evidence.summary.txt
- provider-version-matrix.json

Export adds MANIFEST.json with SHA-256 hashes for every file in the bundle.

 The public bundle references fixture evidence and test-plan shape only. It never carries screenshots, tenant IDs, device IDs, UPNs, Team IDs, or recovery keys.

Export-MacLabEvidence creates a portable directory or zip bundle and adds MANIFEST.json with checksums.

Redaction in Action

Show that the secret exists. Do not show the secret. The write path fails closed if `redactionApplied` is not `true`.

Before Redaction

```
{
  "personalRecoveryKey":
    "ABCD-EFGH-1234-5678",
  "graphToken":
    "eyJhbGc...truncated",
  "payloadUUID":
    "3F2504E0-4F89-11D3",
  "homePath":
    "/Users/jsmith",
  "upn":
    "admin@example.invalid",
  "teamId":
    "Developer ID App: Co (TEAMID1234)"
}
```

After Redaction

```
{
  "personalRecoveryKey":
    "***REDACTED***",
  "graphToken":
    "***REDACTED***",
  "payloadUUID":
    "***REDACTED***",
  "homePath":
    "/Users/***REDACTED***",
  "upn":
    "***REDACTED***",
  "teamId":
    "***REDACTED***"
}
```

`Protect-MacLabEvidence` redacts by field name *and* by shape: recovery-key-shaped strings, JWT-shaped strings, UUIDs, MAC addresses, email addresses, local home paths, and Gatekeeper Team IDs. `Write-EvidenceRecord` re-runs redaction before writing. `Export-MacLabEvidence` re-runs redaction again before packaging.

Appendix: FileVault Evidence Model

Test that escrow exists. Then test it again on real hardware.

1

Policy Assignment

Policy assignment captured and confirmed in evidence bundle.

2

Local State

`fdsetup status` captured locally — not just portal assertion.

3

Encryption / Escrow Report

Intune report confirming encryption and escrow status.

4

Recovery Key Existence

Key existence proven. **Value redacted.**
Never show the key on a projector.

5

Escrow Timing

Escrow preparation and encryption state are not the same thing — test both separately.

6

Hardware Sign-Off Boundary

Documented. VM testing is good for iteration; physical hardware required for final rollout sign-off.

7

Rollback Result

Rollback captured with cloud-state caveat documented in the evidence record.

⚠ Fidelity: ● Yellow. VM iteration is valuable, but physical hardware sign-off is required for the final FileVault rollout validation. Escrow prep and encryption state are separate — test both.

The break-and-rollback workflow you just saw on Gatekeeper is the same workflow you run for a FileVault policy. Same checkpoints. Same evidence shape. Same cloud-state caveat.

Appendix: Defender Evidence Model

Defender on macOS is not an app install. It needs profile deployment, too.

1

Package Installed

App binary present and version recorded.

2

System Extension Approved

System extension loaded and approved via profile delivery — not user prompt.

3

Network Extension Approved

Network extension approved when used. Captured in evidence.

4

Full Disk Access / PPC Delivered

Full Disk Access granted via PPC profile — verified, not assumed.

5

Onboarding Completed

Onboarding blob delivered, device visible in Defender portal.

6

mdatp health

Captured as sanitized key/value output. Do not assume JSON unless the installed version has been verified to emit valid JSON.

7

Rollback Result

Rollback captured. Cloud-state caveat documented. Profiles do not roll back with the VM disk.



Fidelity: ● Green for health checks. ● Yellow for performance-sensitive behavior.

"Looks installed but is not healthy" is the most common Defender macOS failure mode. A misconfigured PPC payload is exactly what the lab catches before production.



The profiles are the deployment. mdatp health is the closest thing to a health receipt. Capture it every run.

WRAP-UP

Wrap-Up: Troubleshooting, the Repo, and Monday Morning.

Appendix: Troubleshooting Checklist

Most macOS Intune failures are APNs, sync, scope, or stale device records. Full table in docs/Troubleshooting.md.

Symptom	Likely Cause	First Check
VM enrolls, policy never arrives	APNs, sync, or assignment scope	Check APNs cert validity; run Company Portal Check Status; verify scope group membership
Firefox stays blocked after rollback	Wrong checkpoint or policy reapplied	Confirm you restored Post-Enroll-Baseline, not Broken-Policy-State
Enrollment fails silently	Stale device record in Intune / Entra	Delete stale record; re-enroll from Pre-Enroll checkpoint
mdatp health shows unhealthy	System extension not approved or PPC not delivered	Verify profile delivery; check System Extensions in System Settings
FileVault not escrowed	Escrow profile timing	Confirm profile delivered before encryption started; check Intune encryption report
Compliance policy not evaluating	Cloud sync not complete	Wait for MDM check-in cycle; run Company Portal Check Status
Provider hardening check fails	Shared profile or SmartMount still enabled	Re-run provider hardening verification after VM creation
Evidence JSON missing redaction	Redaction not applied before write	Confirm redactionApplied: true in output; re-run Protect-MacLabEvidence

The Memorable Anti-Patterns



Portal-Only Evidence

Trusting "assigned" without device-side evidence. The portal reflects intent, not delivery.



Snapshot Time Travel

Rolling back the VM and forgetting the cloud moved forward. Intune, Entra, and Defender do not restore with the disk.



Progress-Bar Theater

Spending stage time watching downloads or cloud sync. Cache the artifact. Pre-stage the cloud thread.



Executive First Test

Using the CEO's Mac as the first real policy test. Production is a terrible place to discover assumptions.



Stale Device Record

A stale Intune/Entra record that no longer matches the VM. Creates invisible compliance gaps and confusing policy behavior.



Screenshot Leak

Showing a recovery key, token, or real user detail because evidence was not redacted. Show that the secret exists. Do not show the secret.

The Repo – On Monday Morning

Repository Structure

```
macOSLab/  
├── src/Modules/MacLab/  
├── scripts/  
├── examples/  
│   ├── MMSMOA-2026/  
│   └── utm/  
├── docs/  
│   ├── Start-Here.md ← start here  
│   └── Troubleshooting.md  
└── tests/
```

Your First Command

```
Import-Module \  
    ./src/Modules/MacLab/MacLab.psd1
```

Five top-level folders. Start at docs/Start-Here.md. The first command imports the module. The second runs the readiness gate. The third pins your media.

- ✓ Starter kit, not finished product. Open issues and pull requests are expected and welcome.

Scan to Clone

<https://github.com/franklesniak/macOSLab>

Public repo · Permissive license · Apple-silicon Macs



- ⓘ Technically, you do not need to remember anything from this talk. You need to remember the repo link. Everything else is in the docs!

The Monday Morning Plan

Do not try to build your whole Mac program by Friday. Pick one risky policy. Run the loop once. The second loop is much easier.



1. Clone the Repo

Start at `docs/Start-Here.md`



3. Take a Pre-Enroll Snapshot

Your enrollment retry point



5. Test One Risky Policy

FileVault, Defender, or Gatekeeper



7. Roll Back

Restore Pre-Enroll or Post-Enroll-Baseline



2. Build One VM

One host, one pinned macOS version



4. Enroll into a Lab Scope

Dedicated lab Intune scope — not production



6. Export Redacted Evidence

The artifact that goes into the change record



8. Review the Cloud Cleanup Report

Report-only — confirm, do not skip

Recap

1

✓ **Analyze Virtualization Constraints & Select Hypervisor**

Analyze the constraints of Apple Silicon and select the appropriate hypervisor that aligns with your budget and automation needs.

Delivered: Part 1 (Constraints) and Part 2 (Hypervisor Decision Matrix).

2

✓ **Construct a Fully Automated, Reproducible Lab**

Construct a fully automated, reproducible macOS test lab using PowerShell 7 to fetch restore images and control VM states.

Delivered: Part 3 (Automation Layer) and Demos 1, 2, and 3.

3

✓ **Execute End-to-End Policy Validation**

Execute end-to-end validation of high-risk policies by enrolling, breaking, and rolling back VMs via script — keeping production safe.

Delivered: Demo 4 (live break and rollback) and FileVault/Defender evidence models.

4

✓ **Implement the GitHub Starter Kit**

Implement the provided GitHub starter kit immediately to bridge the gap between your Windows automation skills and macOS requirements.

Delivered: Windows-admin translation, repo handoff, and the Monday morning plan.

Pin. Build. Break. Roll back. Prove it.

SAVE THE DATES

Oct 25-28, 2026



May 2-6, 2027



Oct 10-13, 2027



Extended Q&A



Recast



ninjaOne®



numecent®

